

The MiniSql Translator

Submitted By

Student Name	Student ID
Muhaiminul Islam Hasin	232-15-667
Md Mehrab Hossen	232-15-754

LAB PROJECT REPORT

Compiler Design Lab in the Department of Computer Science and Engineering



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

14/12/2025

Submitted to:

Chayti Saha

Lecturer,

Department of CSE,

Daffodil International University, Dhaka

Project Title	MiniSQL SELECT-FROM-WHERE Parser
Developed By	[Muhaiminul Islam Hasin, Md Mehrab Hossen]
Technology Stack	C, Flex (Lexical Analyzer Generator), Bison (Parser Generator)
Objective	To design, implement, and validate a robust parser for a restricted subset of SQL SELECT queries, ensuring successful extraction of the Table Name, Column List, and WHERE Condition for subsequent database engine processing.
Version	2.1 (Final Technical Specification)

1. Introduction and Architectural Overview

This project successfully implemented a MiniSQL parser using the industry-standard compiler tools, Flex and Bison. The parser functions as the **Syntactic Analysis** phase of a database front-end, validating the structure of an input query against a predefined Context-Free Grammar (CFG).

The system operates based on the classic compiler pipeline:

System Architecture

2. Implementation Details: Lexical Analysis (minisql.l)

The Flex specification (minisql.l) defines the rules for tokenizing the input stream. This stage correctly distinguishes between reserved keywords, multi-character operators, identifiers, and numerical literals.

2.1 Token Recognition and Data Handling

Token Type	Flex Pattern	Semantic Action	Data Type (%union)
Keywords	"SELECT", "FROM", "WHERE"	Returns fixed token codes.	N/A
Identifiers (ID)	`{letter}({letter}	[0-9_])*`	Uses strdup(yytext) to allocate and pass the string (sval) to the parser.
Numbers (NUMBER)	{digit}+	Uses atoi(yytext) to convert and pass the integer value (ival) to the parser.	int ival
Comparison Ops	"=", "<>", ">=", etc.	Returns specific token codes (EQ, NE, GE, etc.).	N/A
Error Handling	.	Catches undefined characters and prints an "Unknown character" message.	N/A

3. Implementation Details: Syntactic Analysis (minisql.y)

The Bison grammar (minisql.y) dictates the acceptable sequence of tokens and executes semantic actions upon successful rule reduction.

3.1 Context-Free Grammar (CFG)

The primary grammar rules define the structure of the allowed SQL subset:

query SELECT select_list FROM ID opt_where SEMI

select_list STAR | column_list

column_list ID | column_list COMMA ID

OPT_where | WHERE condition

3.2 WHERE Condition Grammar and Precedence

The grammar for the conditional expression handles complexity through recursion and explicit precedence definition:

Condition expr

| **condition AND condition**

| **condition OR condition**

| **LPAREN condition RPAREN**

Expr **ID com_op value**

Precedence and Associativity: Bison's %left and %nonassoc directives correctly resolve conflicts: AND has higher precedence than OR, ensuring boolean logic follows standard rules.

4. Semantic Actions and Memory Management

The most critical function of the semantic actions is the dynamic management of string data, particularly for the potentially complex WHERE clause.

4.1 Data Extraction Strategy

1. **Static Buffers:** Final extracted data is stored in pre-allocated global arrays (current_table, selected_columns, where_clause).
2. **Column List Aggregation:** The recursive column_list rule uses strncat and strncpy to build the full column list string (selected_columns).
3. **WHERE Clause Construction:** This process employs a **bottom-up dynamic allocation strategy**:
 - **Base Case (expr):** Allocates memory for the single comparison string (e.g., col > 10).
 - **Recursive Step (condition AND/OR condition):** Allocates a *new* buffer large enough for the combined string, formats the string with sprintf, and immediately calls free() on the component strings (\$1, \$3) to prevent memory leaks.

This rigorous memory management ensures the parser operates efficiently without memory overhead from temporary string concatenations.

5. Conclusion and Results

The MiniSQL SELECT-FROM-WHERE Parser successfully validates the syntax and extracts the core components of all queries within its defined scope. The project demonstrates effective use of Lex/Flex and Yacc/Bison tools, coupled with disciplined C programming techniques for memory management.

Future Enhancements

- **String Literal Support:** Extend minisql.l to handle quoted string literals (e.g., 'text') and update value rules in minisql.y.
- **Query Optimization:** Replace string-based output with the construction of an Abstract Syntax Tree (AST) for improved downstream processing and query optimization capabilities.

Valid SQL query.

Table : employees

Columns : id, name, salary

WHERE : (salary >= 50000 AND id < 100) OR name = Manager